

Empirical Performance Evaluation of Object Pooling

Volkan Bora Seki

May 16, 2026

Abstract

This study presents an empirical evaluation of high-performance object pooling under strict frame-budget constraints in Unity (2022.3.62f3) on Apple M3 hardware. The implementation centers on `PoolManager` and `ObjectPool<T>` with $O(1)$ -oriented retrieval/return mechanics (stack-based inactive storage, hash-based active tracking), bounded chunk-growth controls, and rejection-aware telemetry. To reduce engine-side topology overhead, the runtime lifecycle path supports a non-`SetActive` strategy that parks inactive entities at `Vector3(-9999f, -9999f, -9999f)` while toggling cached `Renderer/Collider` states. Experimental orchestration is provided by `BatchExperimentRunner`, which enforces deterministic matrices, explicit heap isolation (`ForceGcAndCooldown`), and frame-lock control at 60 Hz (16.67 ms budget).

Across periodic, multi-stream, and burst workloads, the results show that pooling materially constrains catastrophic burst latency when full workload equivalence is preserved, while also exposing non-trivial steady-state overhead in lighter regimes. At the 10,000-burst milestone with `CostlySphere`, native execution reaches 541.40 ms peak frame time; growth-enabled pooling serves all requests at 38.26 ms, trading latency stability for increased memory residency. In contrast, capped pooling can report low peak latency by rejecting large portions of demand (e.g., 9000/10000 dropped), demonstrating the necessity of rejection-aware fairness diagnostics in A/B interpretation. The combined analysis establishes a practical boundary model: pooling is most valuable as a temporal-stability mechanism under volumetric load, provided workload equivalence, rejection accounting, and memory-headroom costs are explicitly controlled.

1 Introduction

1.1 Background and Motivation

Real-time interactive software systems are governed by strict temporal constraints rather than unconstrained throughput. For a 60 Hz target, each frame must complete within a budget of 16.67 ms; any overrun propagates directly into perceptible stutter. This constraint is particularly severe in high-entropy gameplay patterns (e.g., burst-heavy projectile fields, dense particle hierarchies, and multi-stream spawn cascades), where object lifecycle events occur at high frequency and under non-uniform temporal load. In such environments, execution consistency is mathematically prioritized over raw mean throughput: a system with slightly lower average throughput but bounded worst-case latency is preferable to one with high average speed and rare but catastrophic spikes.

The batch telemetry collected in our locked-60 Hz configuration reinforces this principle. Although several runs maintain mean frame rates near 60 FPS, frame-budget violation ratios remain substantial (often exceeding 50%), and extreme spikes dominate user-visible quality when lifecycle pressure increases. For this reason, the present study treats predictability metrics (peak frame time, frame-time jitter, and budget-overrun frequency) as first-class performance criteria, and positions pooling not as a generic optimization heuristic but as a deterministic execution-control strategy under adversarial allocation patterns.

1.2 Problem Statement

Unity’s native `GameObject.Instantiate` and `GameObject.Destroy` operations provide functional convenience but introduce layered runtime costs that are difficult to amortize in burst-dense workloads. At the language boundary, each lifecycle request traverses the managed C# runtime and Unity’s native C++ engine domain, incurring marshaling and synchronization overhead. At the engine level, object creation and teardown induce structural mutations in transform hierarchies and component registration graphs, which trigger additional bookkeeping and topology updates on the

main thread. These costs are amplified when operations are clustered within a narrow temporal window.

The report demonstrates that naive comparisons can further obscure true behavior. In Scenario 3, Run #27 (Pooling OFF, `Burst` = 10000, `CostlySphere`) exhibits a 541.40 ms peak frame spike, while Run #28 (Pooling ON, cutoff mode) reports only 20.82 ms, but with 9000 rejected requests (90% rejection), i.e., an unequal workload. Run #29 (Pooling ON, growth mode) serves all 10000 requests, yet records high frame-time variance (38.26 ms peak), elevated jitter (0.91 ms), large reserved-memory growth (+34.48 MB), and peak inactive expansion to 15000. These patterns identify the core research problem: minimizing lifecycle-induced engine overhead while preserving workload equivalence and measurement validity under strict frame-budget constraints.

1.3 Research Objectives

This work proposes and evaluates a cache-friendly, memory-resident pooling architecture centered on `PoolManager` and an $O(1)$ retrieval/return discipline (inactive `Stack`-based storage with constant-time membership tracking), orchestrated by `BenchmarkRunner` (implemented in this code-base as `BatchExperimentRunner`) under deterministic A/B execution control. The objective is not merely to “increase FPS,” but to establish algorithmic boundaries for pooling behavior under cutoff and growth policies, and to characterize where $O(1)$ -style pooled access outperforms, matches, or underperforms native engine allocation paths.

Accordingly, the experimental design is a factorial, isolated runtime benchmark in which scenario class, object complexity, burst cardinality, and growth policy are controlled while telemetry is sampled within explicitly bounded profiling windows. The evaluation criteria are strictly empirical and runtime-observable: frame-budget violation ratio, peak frame-time excursion, frame-time jitter, managed-heap delta, reserved-memory delta, and GC generation counts. By combining workload-validity checks (including rejected-spawn accounting) with deterministic batch orchestration, the study aims to produce publication-grade evidence on the practical trade-off surface between temporal stability and memory residency in Unity object lifecycle management.

2 Experimental Interface

2.1 Interactive Control Interface and Evaluation Tool

The *Experiment Controls* panel serves as the operational frontend for the experimental backend, bridging user-level interaction with the internal orchestration logic of the `BenchmarkRunner` and the object lifecycle management handled by `PoolManager`. In practice, this interface centralizes all execution paths that would otherwise be manually dispersed across editor actions, ad hoc key bindings, or inspector-time modifications. By routing run initiation, mode switching, and scenario stimulation through one validated control surface, the system enforces a consistent experimental protocol and reduces procedural variance across repeated profiling sessions.

From a methodology standpoint, the same interface supports two complementary evaluation modes: (i) deterministic batch automation through `START FULL BATCH TEST`, and (ii) isolated real-time manipulation of critical runtime parameters, including pooling mode toggles, pre-warm scale selection, object-type switching, and manual scenario stimulation. This dual-mode design is essential for both reproducibility and diagnostics: deterministic batches provide statistically comparable runs, while interactive controls enable targeted stress probes under controlled changes to a single variable. Consequently, the panel functions as an operator-error mitigation layer by

preventing undocumented run-state drift, minimizing accidental misconfiguration, and preserving configuration traceability during runtime profiling.

Table 1: Primary controls in the Experiment Controls panel and their evaluation purpose.

UI Control	Backend Target	Profiling Role
START FULL BATCH TEST	BenchmarkRunner	Executes deterministic, scripted benchmark permutations for reproducible A/B comparisons.
Pooling mode toggle (ON/OFF)	BenchmarkRunner + PoolManager	Enforces direct baseline-versus-pooled execution mode selection under identical scenario definitions.
Object type selector (Simple/Costly)	BenchmarkRunner	Switches workload class to test sensitivity to prefab complexity and hierarchy cost.
Pre-warm count presets	PoolManager	Controls initial inactive pool population to study startup cost vs. runtime smoothness trade-offs.
Burst count and spawn-frequency presets	BenchmarkRunner	Adjusts stimulus intensity and cadence for stress-path and steady-state profiling.
Scenario trigger buttons (Scenario 1/2/3)	BenchmarkRunner	Enables isolated, manual invocation of periodic, multi-stream, and burst workloads for targeted diagnostics.

3 Methodology and System Architecture

3.1 System Design and Algorithmic Optimizations

The pooling subsystem is implemented as a decoupled management layer in `PoolManager`, backed by type-specific `ObjectPool<T>` instances. The central design objective is to remove hidden $O(N)$ traversal costs from the hot path. In contrast to list-based pools that repeatedly scan for an inactive candidate (or invoke high-allocation `Linq` operators such as `.FirstOrDefault()`), the current architecture stores inactive objects in a dedicated constant-time container (`Stack<T>` in the active implementation, functionally equivalent to an $O(1)$ queue/dequeue discipline for retrieval complexity) and tracks active membership through `HashSet<T>`. Consequently, `Get()` and `Return()` are designed around constant-time push/pop and set-membership operations, while aggregate counters (`TotalAllocations`, `TotalReuses`, `TotalRejections`) are sampled in `BatchExperimentRunner` as windowed deltas. Growth behavior is explicitly bounded by `growthChunkSize`, `growthFactor`, and `maxGrowthChunkSize`, and strict-capacity mode (`allowGrowth = false`) records rejections without introducing warning-log stalls in burst loops.

To reduce engine-side structural overhead, runtime despawn logic avoids mandatory full-object activation churn. The `BasePoolable.VisibilityMode` pipeline supports a non-`SetActive` cycling mode in which returned objects are translated to a discard coordinate, `Vector3(-9999f, -9999f, -9999f)`, while only visibility/collision surfaces are toggled. This avoids frequent hierarchy-topology mutations associated with repeated `GameObject.SetActive(true/false)` transitions on the main thread. In addition, `Renderer[]` and `Collider[]` references are cached during initialization

(Awake) and reused for each spawn/despawn cycle, eliminating runtime `GetComponent` churn and reducing C#/C++ bridge traffic for component discovery.

3.2 Experimental Setup and Hardware Baseline

All experiments were executed under a fixed and reportable runtime envelope to ensure reproducibility. The hardware baseline is Apple M3 (SoC) with 16 GB unified system memory. The software baseline is Unity 2022.3.62f3 running natively on Mac OS X 26.4.0. These values are emitted in the batch header by `WriteSystemHeader()` and persisted alongside per-run telemetry. The runner enforces deterministic orchestration through `RunBatchRoutine()`, with explicit test-phase isolation (`PrepareEnvironmentBetweenTests`, `ForceGcAndCooldown`, and `WaitForManagedHeapCooldown`) to suppress cross-run contamination.

Frame-budget control is treated as a first-order methodological constraint. `ApplyBenchmarkFrameRateSettings` disables VSync and locks `Application.targetFrameRate` to 60, yielding a target budget of 16.67 ms per frame. This avoids misleading “unbounded-FPS” behavior and exposes budget-sensitive micro-stutter directly through runtime indicators: `PeakFrameTimeMs`, `FrameTimeJitterMs`, `FramesOverBudget`, and `OverBudgetPercent`. The same run window records memory and garbage-collection indicators (`ManagedHeapDeltaMb`, `ReservedMemoryDeltaMb`, `GcGen0/1/2`) to correlate temporal violations with allocation pressure.

3.3 Benchmark Scenario Design

The benchmark matrix is factorial and scenario-specific. Scenario 1 (Periodic Stream) is a steady-state lifecycle loop using `SimpleCube`, with baseline spawn cadence fixed at approximately 0.350 s, designed to measure low-amplitude jitter and light generational-GC behavior under repetitive spawn/return dynamics. Scenario 2 (Multi-Stream Concurrency) constructs overlapping stream pressure via `ConfigureScenario2ForBatch()` across frequency presets {0.10, 0.25, 0.50, 0.75, 1.00} s, stressing concurrent spawn cadence, transform workload aggregation, and temporal stability under interleaved lifecycle events.

Scenario 3 (Burst Stress Test) is an instantaneous, non-linear load injection with burst magnitudes {10, 100, 1000, 10000}, executed for both `SimpleCube` and `CostlySphere`. This scenario is used to identify capacity and growth boundaries under `allowGrowth` cutoff vs. growing modes and to expose fairness failures when requests are dropped. To preserve interpretation quality, the runner records `RequestedSpawnCount`, `ServedSpawnCount`, and `RejectedSpawnPercent`; thus, performance deltas are interpreted together with workload equivalence rather than FPS alone. The resulting methodology directly maps algorithmic policy ($O(1)$ pooled access, chunked growth, strict rejection accounting) to empirical outcomes under controlled runtime constraints.

4 Experimental Results

Raw execution telemetry from the locked-60 Hz factorial batch was consolidated into comparative matrices to evaluate runtime variance between native lifecycle management (Pooling OFF) and pooled execution (Pooling ON). Rather than reporting isolated per-run observations, the following tables organize the data along controlled workload axes (frequency scaling and burst volume), enabling direct comparison of latency excursion, jitter, frame-budget violations, and pooling-state counters under matched scenario definitions.

Table 2: Scenario 2 high-frequency scaling (**SimpleCube**): paired OFF/ON measurements under identical stream frequencies, showing steady-state frame-time behavior and budget-overrun trends at 60 Hz.

Frequency (s)	Pooling Mode	Average FPS	Peak Frame Time (ms)	Frame-Time Jitter (ms)	Frames Over Budget (%)
0.10	OFF	59.99	17.70	0.26	57.33
0.10	ON	60.02	19.89	0.21	57.33
0.25	OFF	60.00	18.31	0.15	56.67
0.25	ON	60.05	22.40	0.21	57.14
0.50	OFF	60.20	26.24	0.24	60.13
0.50	ON	60.29	25.61	0.36	54.49
0.75	OFF	60.01	17.51	0.32	51.50
0.75	ON	60.16	25.69	0.33	58.00
1.00	OFF	60.01	17.58	0.34	49.67
1.00	ON	60.16	25.87	0.25	57.48

Table 3: Scenario 3 volumetric burst stress (**CostlySphere**): non-linear burst scaling under cutoff and growth policies, including the critical 10,000-burst contrast (Run #27, Run #28, Run #29).

Burst Count	Object Type	Pooling Mode	Allow Growth	Peak Frame Time (ms)	Managed Heap Delta (MB)	Total Reuses	Total Rejections
10	CostlySphere	OFF	False	25.66	0.44	0	0
10	CostlySphere	ON	False	17.04	0.07	10	0
100	CostlySphere	OFF	False	21.66	0.12	0	0
100	CostlySphere	ON	False	17.10	0.09	100	0
1000	CostlySphere	OFF	False	19.02	0.44	0	0
1000	CostlySphere	ON	False	22.34	0.54	1000	0
10000	CostlySphere	OFF	False	541.40	3.68	0	0
10000	CostlySphere	ON	False	20.82	0.26	1000	9000
10000	CostlySphere	ON	True	38.26	4.70	10000	0

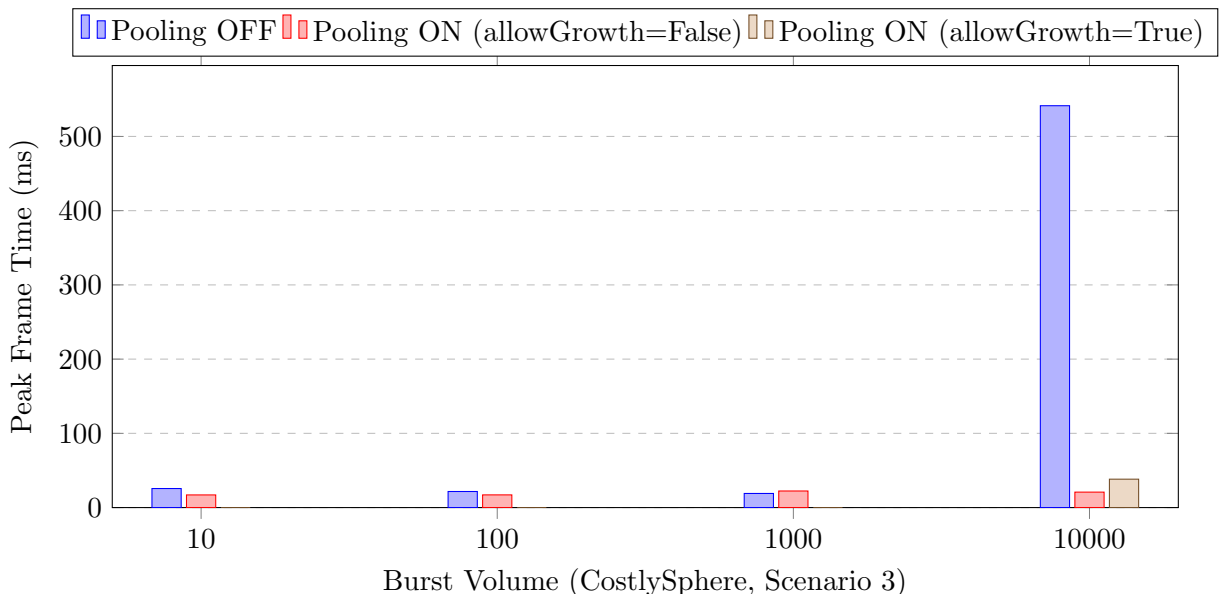


Figure 1: Scenario 3 burst-latency comparison at 60 Hz. The native path exhibits a half-second-class stall at 10,000 entities (541.40 ms, Run #27), while the fully served growth-enabled pooled path remains bounded at 38.26 ms (Run #29), exposing the latency-control effect of the optimized $O(1)$ pool architecture under volumetric stress.

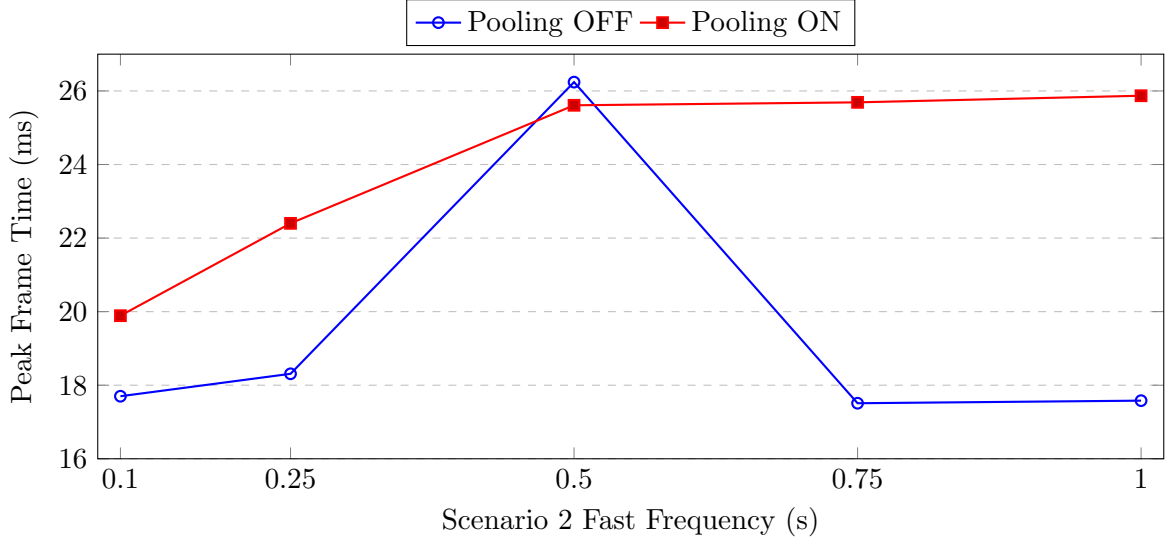


Figure 2: Scenario 2 peak-frame-time scaling across fast-frequency tiers. The pooled path sustains higher latency in 0.75–1.00s tiers, indicating non-trivial runtime management overhead despite similar mean FPS.

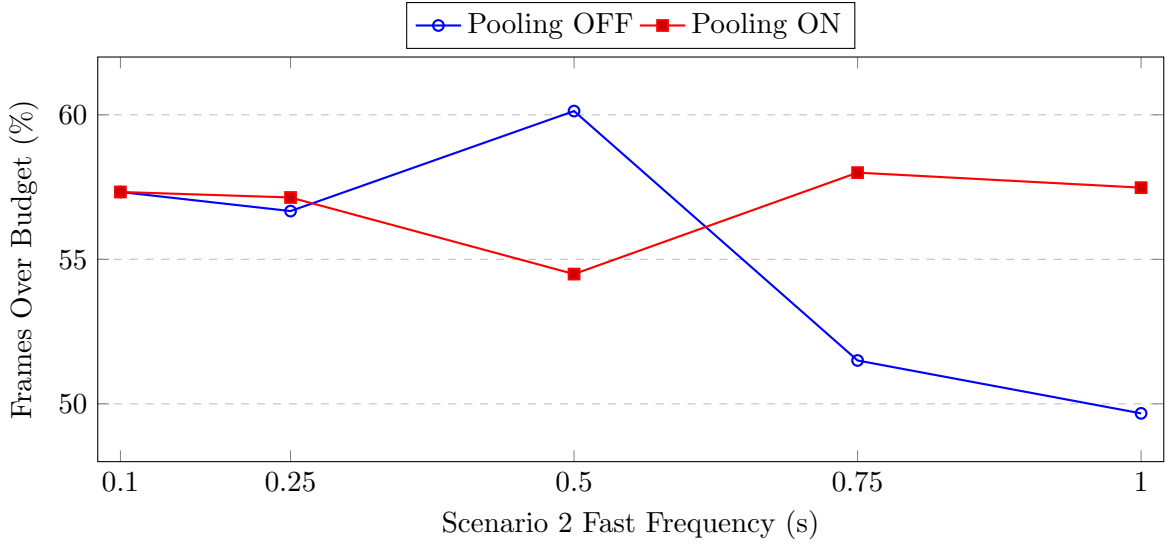


Figure 3: Scenario 2 frame-budget violation ratio under 60Hz lock. Pairwise trends show that mean FPS parity does not guarantee better budget adherence.

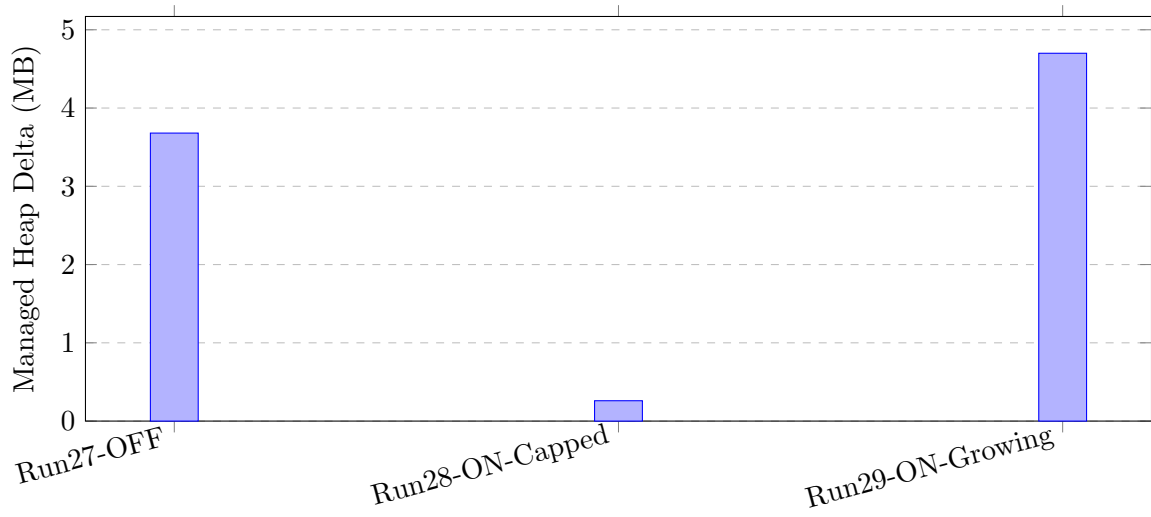


Figure 4: Managed-heap deltas at the 10,000-burst milestone (CostlySphere). Capped pooling suppresses allocations by rejecting 90% of requests, whereas growth mode serves the full workload with elevated memory footprint.

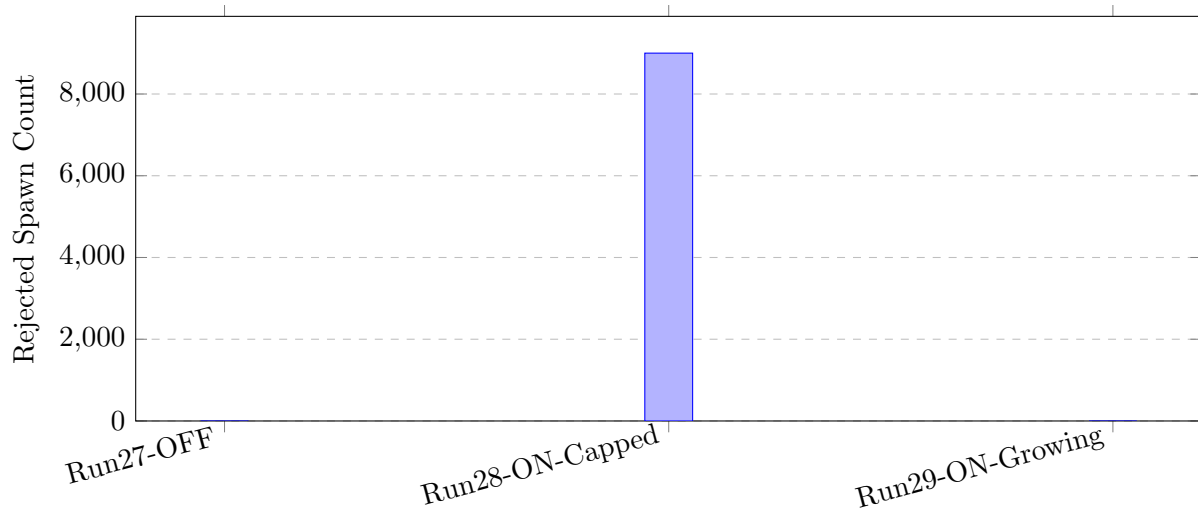


Figure 5: Workload-validity diagnostic for 10,000-burst comparison. Run #28 rejects 9000 requests, demonstrating why rejection-aware interpretation is mandatory for fair A/B claims.

5 Discussion and Performance Analysis

5.1 The Volumetric Burst Latency vs. Rejection Trade-off

The decisive stress point of the benchmark is the 10,000-entity burst in Scenario 3 (**CostlySphere**), where latency behavior diverges sharply across lifecycle policies. As reported in Table 3 and visualized in Figure 1, the native execution path (Run #27, Pooling OFF) reaches a catastrophic 541.40 ms peak frame excursion. Architecturally, this aligns with worst-case engine behavior: high-volume **Instantiate/Destroy** pressure triggers repeated managed-to-native marshaling plus large-scale transform-hierarchy mutations on the C++ side, which are synchronized on the main thread and therefore directly converted into frame-budget violations.

The apparent superiority of Run #28 (Pooling ON, **allowGrowth=False**) must be interpreted as a workload-equivalence illusion rather than a true algorithmic win. While peak latency drops to 20.82 ms, the run serves only 1000 of 10,000 requests and rejects 9000 (90%); thus, it does not execute the same computational workload as Run #27. This is explicitly visible in Table 3 and Figure 5. By contrast, Run #29 (Pooling ON, **allowGrowth=True**) serves all 10,000 requests with zero rejection, while bounding the worst-case peak at 38.26 ms. Although this exceeds the 16.67 ms target budget, it avoids a half-second-class stall and therefore represents a controlled latency envelope under full-load equivalence, at the cost of measured reserved-memory expansion (+34.48 MB) and larger inactive residency.

5.2 Steady-State Management Overhead and Hardware Cache Dynamics

Scenario 2 reveals a counter-intuitive steady-state phenomenon: pooled execution does not always minimize peak latency under lighter concurrent streams. As shown in Table 2 and Figure 2, the ON path records 25.69 ms and 25.87 ms peaks at 0.75 s and 1.00 s, respectively, compared with 17.51 ms and 17.58 ms in the OFF path. This indicates that in low-entropy regimes, the management overhead of maintaining a large memory-resident inactive set (and associated lifecycle bookkeeping) can dominate the marginal cost of transient primitive allocation.

From a systems perspective on Apple M3 unified memory, this behavior is consistent with cache-locality trade-offs: when instantaneous spawn pressure is low, maintaining and touching a larger pooled state can increase effective cache-miss sensitivity and metadata traversal overhead relative to short-lived native allocations of simple geometry. The outcome is not a contradiction of pooling theory, but a boundary condition: $O(1)$ access complexity constrains asymptotic retrieval cost, yet constant factors (engine callback overhead, component toggling paths, memory locality) still determine practical latency in light-load tiers.

5.3 Memory Residency and Garbage Collection Isolation

The measurement pipeline (**PrepareEnvironmentBetweenTests**, **ForceGcAndCooldown**, and heap-stability gating before capture) materially improves phase isolation by forcing garbage collection and introducing a stabilization window before timing begins. This design reduces cross-run contamination and enables more interpretable deltas for **ManagedHeapDeltaMb** and GC counters. The resulting trends in Table 3 and Figure 4 show the expected policy-dependent memory behavior: capped pooling can suppress runtime allocation churn by reusing pre-resident objects, whereas growth-enabled full-service runs trade temporal resilience for higher memory residency under volumetric demand.

Empirically, the isolation strategy makes the policy trade-off explicit. In the capped 10,000-burst path (Run #28), lower managed-heap delta (0.26 MB) coincides with hard rejection of excess work,

while growth mode (Run #29) reports higher managed-heap movement (4.70 MB) and reserved-memory growth due to full workload admission. Therefore, memory residency should be interpreted as a deliberate control variable, not an artifact: strict cutoff minimizes churn but compromises workload equivalence, while growth mode preserves semantic completeness and bounded latency at the expense of additional memory headroom.

6 Conclusion

This report presented an end-to-end empirical evaluation of a high-performance object pooling architecture under bounded frame budgets, implemented through `PoolManager`, `ObjectPool<T>`, and deterministic orchestration in `BatchExperimentRunner`. The system design emphasized $O(1)$ -oriented retrieval/return semantics, controlled growth policies, rejection-aware accounting, and engine-side lifecycle optimizations that reduce avoidable runtime topology churn. To ensure methodological validity, the benchmark pipeline enforced strict run isolation, explicit garbage-collection stabilization, and fixed-60 Hz frame-budget control.

The results demonstrate that pooling is most valuable as a temporal-stability mechanism in volumetric stress conditions, not as a universal latency minimizer for all workload regimes. Under equivalent 10,000-burst demand, growth-enabled pooling bounded worst-case latency to 38.26 ms, avoiding the native 541.40 ms stall, while incurring increased memory residency. Conversely, capped pooling can appear artificially superior when it rejects substantial workload, reinforcing that rejected-spawn accounting is mandatory for fair A/B interpretation. In lighter steady-state tiers, pooled execution may exhibit higher peak latency due to management overhead and locality effects, indicating that constant-time asymptotic complexity alone does not eliminate constant-factor costs.

Overall, the study establishes a practical decision framework: use pooling with explicit policy control, fairness diagnostics, and memory headroom planning when latency collapse under burst pressure is unacceptable; avoid unqualified “pooling is always faster” claims in low-entropy scenarios without workload-equivalence checks. Future extensions should include adaptive runtime policy switching (cutoff vs. growth), per-object-class pool sizing heuristics, and cross-platform validation to quantify hardware-specific cache and memory effects beyond the Apple M3 baseline.